

Création API REST

Voici les fichiers présents dans le dossier hippodrome_api :

Src :

- Arrosage

Arrosage.controller.ts :

```
import { Controller, Get, Body, Post } from '@nestjs/common';
import { ArrosageService } from './arrosage.service';
import { Arrosage } from './arrosage.entity';
```

```
@Controller('arrosage')
```

```
export class ArrosageController {
  constructor(private readonly arrosageService: ArrosageService) {}
```

```
  @Get()
```

```
  findAll(): Promise<Arrosage[]> {
    return this.arrosageService.findAll();
  }
```

```
  @Post()
```

```
  create(@Body() arrosage: Arrosage): Promise<Arrosage> {
    return this.arrosageService.create(arrosage);
  }
}
```

Arrosage.entity.ts :

```
import { Entity, PrimaryGeneratedColumn, Column, ManyToOne, JoinColumn }
from "typeorm";
import { Pistes } from "src/pistes/pistes.entity";
```

```
@Entity()
```

```
export class Arrosage {
  @PrimaryGeneratedColumn()
  id_arrosage: number;
```

```
  @Column()
```

```
  horodatage: Date;
```

```
  @Column()
```

```
  quantite:number;
```

```
  @ManyToOne(() => Pistes, (pistes) => pistes.arrosage)
```

```
  @JoinColumn()
```

```
    piste: Pistes;
  }
```

```
Arrosage.module.ts :
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { Arrosage } from './arrosage.entity';
import { ArrosageService } from './arrosage.service';
import { ArrosageController } from './arrosage.controller';
```

```
@Module({
  imports: [TypeOrmModule.forFeature([Arrosage])],
  providers: [ArrosageService],
  controllers: [ArrosageController]
})
export class ArrosageModule {}
```

```
Arrosage.service.ts :
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Arrosage } from './arrosage.entity';
```

```
@Injectable()
export class ArrosageService {
  constructor(
    @InjectRepository(Arrosage)
    private ArrosageRepository: Repository<Arrosage>,
  ){}

  findAll(): Promise<Arrosage[]>{
    return this.ArrosageRepository.find({ relations: ['piste']});
  }

  create(arrosage: Arrosage): Promise<Arrosage> {
    return this.ArrosageRepository.save(arrosage);
  }
}
```

- Capteur

```
Capteur.controller.ts :

import { Controller, Get, Post, Body } from '@nestjs/common';
import { CapteurService } from './capteur.service';
import { Capteur } from './capteur.entity';
```

```

@Controller('capteur')
export class CapteurController {
  constructor(private readonly capteurService: CapteurService) {}

  @Get()
  findAll(): Promise<Capteur[]> {
    return this.capteurService.findAll();
  }

  @Post()
  create(@Body() capteur: Capteur): Promise<Capteur> {
    return this.capteurService.create(capteur);
  }
}

```

Capteur.entity.ts :

```

import { Entity, PrimaryGeneratedColumn, Column, ManyToOne, OneToMany,
JoinColumn } from "typeorm";

import { Hippodrome } from "src/hippodrome/hippodrome.entity";
import { MesuresMeteo } from "src/mesures_meteo/mesures_meteo.entity";
import { MesuresTerrain } from "src/mesures_terrain/mesures_terrain.entity";

```

```

@Entity()
export class Capteur {
  @PrimaryGeneratedColumn()
  id_capteur: number;

  @ManyToOne(() => Hippodrome, (hippodrome) => hippodrome.capteurs)
  @JoinColumn({ name: 'id_hippodrome' })
  hippodrome: Hippodrome;

  @Column()

```

```

    type_capteur: string;

    @Column()
    zone: number;

    @OneToMany(() => MesuresMeteo, (measure) => measure.capteur)
    mesuresMeteo: MesuresMeteo[];

    @OneToMany(() => MesuresTerrain, (measure) => measure.capteur)
    mesuresTerrain: MesuresTerrain[];
}

```

Capteur.module.ts :

```

import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { Capteur } from './capteur.entity';
import { CapteurService } from './capteur.service';
import { CapteurController } from './capteur.controller';

```

```

@Module({
  imports: [TypeOrmModule.forFeature([Capteur])],
  providers: [CapteurService],
  controllers: [CapteurController]
})
export class CapteurModule {}

```

Capteur.service.ts :

```

import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Capteur } from './capteur.entity';
import { Repository } from 'typeorm';

```

```

@Inject()
export class CapteurService {
  constructor(
    @InjectRepository(Capteur)
    private CapteurRepository: Repository<Capteur>,
  ) {}

  findAll(): Promise<Capteur[]> {
    return this.CapteurRepository.find({ relations: ['hippodrome']});
  }

  create(capteur: Capteur): Promise<Capteur> {
    return this.CapteurRepository.save(capteur);
  }
}

```

- Hippodrome

```

Hippodrome.controller.ts :
import { Controller, Get, Post, Body } from '@nestjs/common';
import { HippodromeService } from './hippodrome.service';
import { Hippodrome } from './hippodrome.entity';

@Controller('hippodrome')
export class HippodromeController {
  constructor(private readonly hippodromeService: HippodromeService) {}

  @Get()
  findAll(): Promise<Hippodrome[]> {
    return this.hippodromeService.findAll();
  }

  @Post()
  create(@Body() Hippodrome: Hippodrome): Promise<Hippodrome> {
    return this.hippodromeService.create(Hippodrome);
  }
}

```

Hippodrome.entity.ts :

```

import { Entity, PrimaryGeneratedColumn, Column, OneToMany } from
"typeorm";
import { Capteur } from 'src/capteur/capteur.entity';
import { Pistes } from "src/pistes/pistes.entity";

@Entity()
export class Hippodrome {
  @PrimaryGeneratedColumn()
  id_hippodrome: number;

  @Column()
  nom: string;

  @Column()
  ville: string;

  @Column('float')
  superficie: number;

  @Column('int')
  nb_pistes:number;

  @OneToMany(() => Capteur, (capteur) => capteur.hippodrome)
  capteurs: Capteur[];

  @OneToMany(() => Pistes, (piste) => piste.hippodrome)
  pistes: Pistes[];
}

```

Hippodrome.module.ts :

```

import { Module } from '@nestjs/common';
import { HippodromeService } from './hippodrome.service';
import { HippodromeController } from './hippodrome.controller';
import { TypeOrmModule } from '@nestjs/typeorm';
import { Hippodrome } from './hippodrome.entity';

```

```

@Module({
  imports: [TypeOrmModule.forFeature([Hippodrome])],
  providers: [HippodromeService],
  controllers: [HippodromeController]
})
export class HippodromeModule {}

```

Hippodrome.service.ts :

```

import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Hippodrome } from './hippodrome.entity';

```

```

@Injectable()
export class HippodromeService {
  constructor(
    @InjectRepository(Hippodrome)
    private HippodromeRepository: Repository<Hippodrome>,
  ) {}

  findAll(): Promise<Hippodrome[]> {
    return this.HippodromeRepository.find();
  }

  create(hippodrome: Hippodrome): Promise<Hippodrome> {
    return this.HippodromeRepository.save(hippodrome)
  }
}

```

- Mesures_meteo

```

Mesures_meteo.controller.ts :
import { Controller, Get, Param } from '@nestjs/common';
import { MesuresMeteoService } from './mesures_meteo.service';
import { MesuresMeteo } from './mesures_meteo.entity';

```

```

@Controller('mesures-meteo')
export class MesuresMeteoController {
  constructor(private readonly service: MesuresMeteoService) {}

  @Get('type/:type')
  async findByType(@Param('type') type: string) {
    return this.service.findByType(type);
  }
}

```

```

Mesures_meteo.entity.ts :
import { Entity, PrimaryGeneratedColumn, Column, ManyToOne, JoinColumn }
from "typeorm";
import { Capteur } from "src/capteur/capteur.entity";

```

```

@Entity()
export class MesuresMeteo {
  @PrimaryGeneratedColumn()
  id_meteo: number;

  @ManyToOne(() => Capteur, (capteur) => capteur.mesuresMeteo)
  @JoinColumn({ name: 'id_capteur' })
  capteur: Capteur;
}

```

```

@Column()
horodatage: Date;

@Column('float')
temperature_air: number;

@Column('float')
temperature_air_moyenne: number;

@Column('float')
hygrometrie: number;

@Column('float')
hygrometrie_moyenne: number;

@Column('float')
vent_vitesse: number;

@Column('float')
vent_vitesse_moyenne: number;

@Column('int')
vent_direction: number;

@Column('float')
pluviometrie: number;

@Column('float')
ensoleillement: number;

@Column('float')
ensoleillement_moyenne: number;
}

```

Mesures_meteo.module.ts :

```

import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { MesuresMeteo } from './mesures_meteo.entity';
import { MesuresMeteoService } from './mesures_meteo.service';
import { MesuresMeteoController } from './mesures_meteo.controller';

```

```

@Module({
  imports: [TypeOrmModule.forFeature([MesuresMeteo])],
  providers: [MesuresMeteoService],
  controllers: [MesuresMeteoController]
})
export class MesuresMeteoModule {}

```



```

Mesures_meteo.service.ts :
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { MesuresMeteo } from './mesures_meteo.entity';

@Injectable()
export class MesuresMeteoService {
  constructor(
    @InjectRepository(MesuresMeteo)
    private mesuresMeteoRepository: Repository<MesuresMeteo>
  ){}

  async findByType(type: string): Promise<any[]> {
    const all = await this.mesuresMeteoRepository.find();

    const mapValue = (record: MesuresMeteo, value: number, extra?:
Record<string, any>) => ({
      valeur: value,
      date: record.horodatage,
      ...(extra || {}),
    });

    switch (type) {
      case 'TemperatureAir':
        return all.map((m) => mapValue(m, m.temperature_air));
      case 'TemperatureAirMoyenne':
        return all.map((m) => mapValue(m, m.temperature_air_moyenne));
      case 'Hygrométrie':
        return all.map((m) => mapValue(m, m.hygrometrie));
      case 'HygrométrieMoyenne':
        return all.map((m) => mapValue(m, m.hygrometrie_moyenne));
      case 'Pluviometrie':
        return all.map((m) => mapValue(m, m.pluviometrie));
      case 'Ensoleillement':
        return all.map((m) => mapValue(m, m.enseillement));
      case 'EnsoleillementMoyenne':
        return all.map((m) => mapValue(m, m.enseillement_moyenne));
      case 'DirectionVent':
        return all.map((m) => mapValue(m, m.vent_direction));
      case 'VitesseVent':
        return all.map((m) => mapValue(m, m.vent_vitesse));
      case 'VitesseVentMoyenne':
        return all.map((m) => mapValue(m, m.vent_vitesse_moyenne));
      default:
        return [];
    }
  }
}

```

```
}
```

- Mesures_terrain

Mesures_terrain.controller.ts :

```
import { Controller, Get, Param } from '@nestjs/common';
import { MesuresTerrain } from './mesures_terrain.entity';
import { MesuresTerrainService } from './mesures_terrain.service';
```

```
@Controller('mesures-meteo')
```

```
export class MesuresTerrainController {
  constructor(private readonly service: MesuresTerrainService) {}
```

```
  @Get('type/:type')
```

```
  async findByType(@Param('type') type: string) {
    return this.service.findByType(type);
  }
```

```
}
```

Mesures_terrain.entity.ts :

```
import { Entity, PrimaryGeneratedColumn, Column, ManyToOne, JoinColumn }
from "typeorm";
import { Capteur } from "src/capteur/capteur.entity";
```

```
@Entity()
```

```
export class MesuresTerrain {
  @PrimaryGeneratedColumn()
  id_terrain: number;
```

```
  @ManyToOne(() => Capteur, (capteur) => capteur.mesuresMeteo)
```

```
  @JoinColumn({ name: 'id_capteur'})
  capteur: Capteur;
```

```
  @Column()
```

```
  horodatage: Date;
```

```
  @Column('float')
```

```
  profondeur_peneration: number;
```

```
  @Column('float')
```

```
  temperature_sol: number;
```

```
  @Column('float')
```

```
  humidite_sol: number;
```

```
}
```

Mesures_terrain.module.ts :

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
```

```
import { MesuresTerrain } from './mesures_terrain.entity';
import { MesuresTerrainService } from './mesures_terrain.service';
import { MesuresTerrainController } from './mesures_terrain.controller';
```

```
@Module({
  imports: [TypeOrmModule.forFeature([MesuresTerrain])],
  providers: [MesuresTerrainService],
  controllers: [MesuresTerrainController]
})
export class MesuresTerrainModule {}
```

Mesures_terrain.service.ts :

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { MesuresTerrain } from './mesures_terrain.entity';
```

```
@Injectable()
export class MesuresTerrainService {
  constructor(
    @InjectRepository(MesuresTerrain)
    private mesuresTerrainRepository: Repository<MesuresTerrain>
  ) {}
```

```
  async findByType(type: string): Promise<any[]> {
    const all = await this.mesuresTerrainRepository.find({relations: ['capteur'] });
```

```

    const mapValue = (record: MesuresTerrain, value: number) => ({
      valeur: value,
      date: record.horodatage,
      zone: record.capteur?.zone || 'inconnue',
    });
  }
```

```
  switch (type) {
    case 'ProfondeurPenetration':
      return all.map((m) => mapValue(m, m.profondueur_peneration));
    case 'TemperatureSol':
      return all.map((m) => mapValue(m, m.temperature_sol));
    case 'HumiditeSol':
      return all.map((m) => mapValue(m, m.humidite_sol));
    default:
      return [];
  }
}
```

- Pistes

Pistes.controller.ts :

```
import { Controller, Get, Post, Body } from '@nestjs/common';
import { PistesService } from './pistes.service';
import { Pistes } from './pistes.entity';
```

```
@Controller('pistes')
export class PistesController {
  constructor(private readonly pistesService: PistesService) {}

  @Get()
  findAll(): Promise<Pistes[]> {
    return this.pistesService.findAll();
  }

  @Post()
  create(@Body() pistes: Pistes): Promise<Pistes> {
    return this.pistesService.create(pistes);
  }
}
```

Pistes.entity.ts :

```
import { Entity, PrimaryGeneratedColumn, Column, ManyToOne, JoinColumn,
OneToMany } from "typeorm";
import { Hippodrome } from "src/hippodrome/hippodrome.entity";
import { Arrosage } from 'src/arrosage/arrosage.entity'
```

```
@Entity()
export class Pistes {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  numero_piste: string;

  @ManyToOne(() => Hippodrome, (hippodrome) => hippodrome.nb_pistes)
  @JoinColumn({ name: 'id_hippodrome' })
  hippodrome:Hippodrome;

  @OneToMany(() => Arrosage, (arrosage) => arrosage.piste)
  arrosage: Arrosage[];
}
```

Pistes.module.ts :

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { Pistes } from './pistes.entity';
import { PistesService } from './pistes.service';
import { PistesController } from './pistes.controller';
```

```

@Module({
  imports: [TypeOrmModule.forFeature([Pistes])],
  providers: [PistesService],
  controllers: [PistesController]
})
export class PistesModule {}

```

Pistes.service.ts :

```

import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Pistes } from './pistes.entity';

```

```

@Injectable()
export class PistesService {
  constructor(
    @InjectRepository(Pistes)
    private PistesRepository: Repository<Pistes>,
  ) {}

  findAll(): Promise<Pistes[]>{
    return this.PistesRepository.find({ relations: ['hippodrome', 'arrosage']});
  }

  create(pistes: Pistes): Promise<Pistes> {
    return this.PistesRepository.save(pistes);
  }
}

```

App.controller.ts :

```

import { Controller, Get } from '@nestjs/common';
import { AppService } from './app.service';

```

```

@Controller()
export class AppController {

  constructor(private readonly appService: AppService) {}

```

```

@Get()
getHello(): string {

  return this.appService.getHello();
}

```

```
}
```

App.module.ts :

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { AppController } from './app.controller';
import { AppService } from './app.service';
import { HippodromeModule } from './hippodrome/hippodrome.module';
import { CapteurModule } from './capteur/capteur.module';
import { MesuresMeteoModule } from './mesures_meteo/mesures_meteo.module';
import { MesuresTerrainModule } from './mesures_terrain/mesures_terrain.module';
import { ArrosageModule } from './arrosage/arrosage.module';
import { PistesModule } from './pistes/pistes.module';
import { Hippodrome } from './hippodrome/hippodrome.entity';
import { Capteur } from './capteur/capteur.entity';
import { MesuresMeteo } from './mesures_meteo/mesures_meteo.entity';
import { MesuresTerrain } from './mesures_terrain/mesures_terrain.entity';
import { Pistes } from './pistes/pistes.entity';
import { Arrosage } from './arrosage/arrosage.entity';
```

```
@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'postgres',
      host: 'localhost',
      port: 5432,
      username: 'postgres',
      password: 'password',
      database: 'hippodrome_db',
      entities: [Hippodrome, Capteur, MesuresMeteo, MesuresTerrain, Pistes, Arrosage],
      synchronize: false,
```

```
    }},  
    HippodromeModule,  
    CapteurModule,  
    MesuresMeteoModule,  
    MesuresTerrainModule,  
    ArrosageModule,  
    PistesModule],  
  })  
  export class AppModule {}
```

App.service.ts :

```
import { Injectable } from '@nestjs/common';
```

```
@Injectable()
```

```
export class AppService {  
  getHello(): string {  
    return 'Hello World!';  
  }  
}
```

Main.ts :

```
import { NestFactory } from '@nestjs/core';  
import { AppModule } from './app.module';  
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';
```

```
async function bootstrap() {  
  const app = await NestFactory.create(AppModule);  
  
  const config = new DocumentBuilder()  
    .setTitle('API Hippodrome')  
    .setDescription('API pour la gestion des capteurs et des mesures')
```

```
.setVersion('1.0')  
  
.build();  
  
const document = SwaggerModule.createDocument(app, config);  
SwaggerModule.setup('api', app, document);  
  
await app.listen(3000);  
}  
bootstrap();
```